Laboratory 1: Short project

Docker, Compose and Swarm — high availability edition

Marc De Rialp David Franco Omar Antonio Cornejo David Madrid
Victor Yakovlev

Cloud Computing and Distributed Systems
MSc in Machine Learning and Cybersecurity for Internet Connected Systems, UPC

October 2026

Abstract

We containerised the digital infrastructure of a company with Docker: a shared stack (web front, certificates, shared disk), a reviews application with its database, cache, mail relay and backups, a monitoring platform, and the Moodle learning management system. The same project runs in two environments: a local one on a single machine (`compose.yml`) and a production one on a Docker Swarm of seven nodes (`production.yml`), in which every user-facing service is replicated, the database and the cache are clustered and fail over by themselves, and the backups are encrypted and kept on a node of their own. The images are published in Docker Hub, so deploying only needs to pull and start them. The report describes the design, the deployment, the management of data and backups, the monitoring, the test cases and their results.

Contents

1 Introduction **3**

2 Architecture **3**
2.1 Nodes . . . 3
2.2 Services . . . 3

3 Images and design decisions **4**

4 Scalability and high availability **5**
4.1 Replicas . . . 5
4.2 Failures . . . 6
4.3 Updates . . . 6

5 Deployment **7**
5.1 Local environment (test) . . . 7
5.2 Production environment (simulated Swarm) . . . 7

6 Access, Users and Passwords **9**
6.1 Web Addresses . . . 9
6.2 Users and Passwords . . . 9

7 Data Management and Backups **10**

8 Observability **11**

9 Security **11**

10 Test cases **12**
10.1 Local environment . . . 12
10.2 Production environment . . . 13

11 Results **14**
11.1 Local environment . . . 14
11.2 Production environment (`production.yml`) . . . 15

12 Limitations **15**

13 Conclusions **15**

1 Introduction

We containerised the infrastructure outlined in the laboratory assignment: the shared services (*basic*), a company application (*app*), an observability platform (*stats*), and Moodle (*atenea*). For the core application, we developed a reviews service in Flask: users can post comments with a 1-to-5 rating, view score distributions and recent reviews, trigger email notifications through an internal SMTP relay upon each submission, and ingest external ratings via an automated background feeder.

The implementation is designed for production-grade resilience: every user-facing component maintains at least two replicas distributed across distinct nodes, the application database and caching layers are clustered with automatic failover, backups are encrypted and stored on a dedicated node, and all services export health and telemetry metrics. Deployment is fully automated, requiring only a single `docker compose up` for local execution or `docker stack deploy` for production Swarm orchestration, using pre-built images published to Docker Hub.

The repository is organised into `compose.yml` (local), `production.yml` (Swarm), service directories containing their respective `Dockerfile` definitions and runtime assets, helper automation in `scripts/` (node labelling, image publishing, Swarm simulation, and disaster recovery), and this report. The local environment operates without TLS and uses named volumes, yet preserves the exact clustering topology present in production to enable verification on a single development machine.

2 Architecture

2.1 Nodes

The swarm comprises seven nodes; service placement is controlled via the Docker Swarm node label `role` (along with `slot` on the application nodes to enforce separation between primaries and replicas).

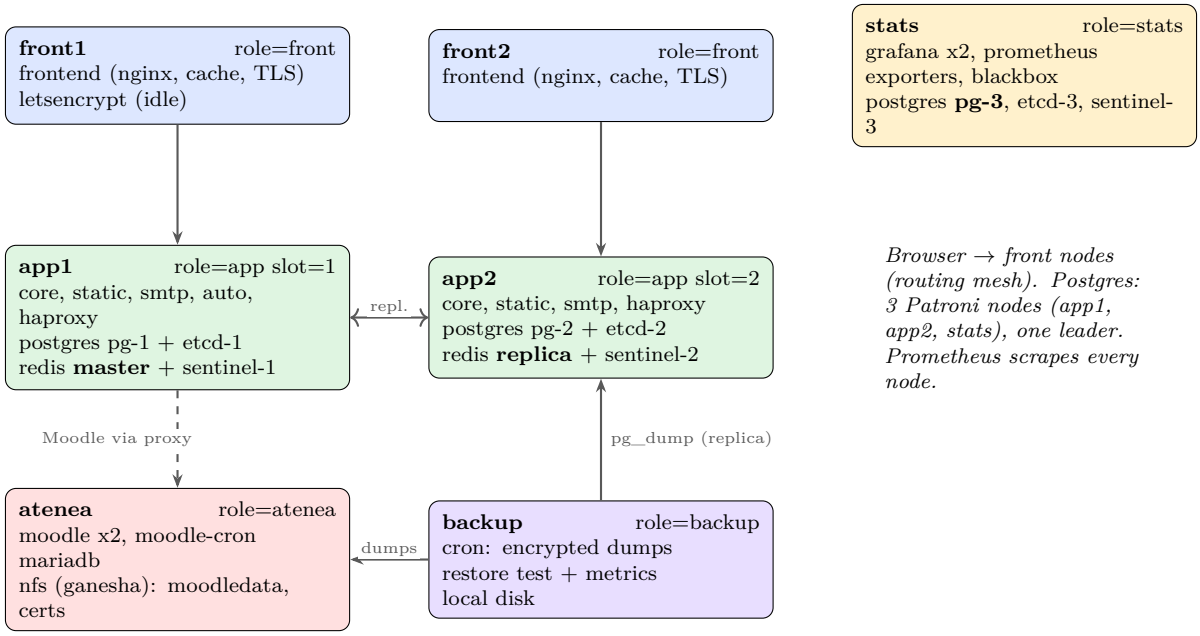


Figure 1: Nodes and services (simplified). The routing mesh accepts the traffic on both front nodes; `backup` pulls from a replica, from MariaDB and from the NFS volume.

2.2 Services

Table 1 maps the components from the assignment specification to our implemented services, the nodes where they run, and their replica counts. Each stack operates within an encrypted overlay

network, and each service only joins the networks it strictly requires: for instance, the databases do not participate in the `proxy` network, ensuring they remain isolated and inaccessible from the frontend.

Stack	Service	Nodes	Repl.	Description
basic	`frontend`	front	2	nginx: only public service (80/443), cache, TLS, proxy by subdomain
	`letsencrypt`	front	1	certbot; idle with a local domain (self-signed certificate)
	`nfs`	atenea	1	NFSv4 server (NFS-Ganesha) inside the stack, port 2049 in host mode
app	`core`	app	2	Flask + gunicorn, reviews
	`static`	app	2	Apache httpd, `/static/`
	`pg-1..3`	app1, app2, stats	3	PostgreSQL 16 + Patroni (leader + 2 replicas)
	`etcd-1..3`	app1, app2, stats	3	consensus store of Patroni
	`db`	app	2	HAProxy: `db:5432` leader, `db:5433` replicas
	`redis-1`, `redis-2`	app1, app2	1+1	cache: master and replica
	`sentinel-1..3`	app1, app2, stats	3	failover of the cache (quorum 2)
	`smtp`	app	2	Postfix relaying the mails (to Mailpit by default)
	`mailpit`	app	1	mail catcher with a web page (`mail.localhost:8080`)
	`auto`	app	1	feeder of external ratings
	`backup`	backup	1	encrypted dumps + restore test
atenea	`moodle`	atenea	2	Moodle 4.5.14 LTS
	`moodle-cron`	atenea	1	same image, `cron.php` every minute
	`moodle-db`	atenea	1	MariaDB 11.4
stats	`grafana`	stats	2	dashboards as code + Infinity plugin (the "?")
	`prometheus`	stats, backup	2	metrics and alert rules, one TSDB per replica
	exporters	stats / global	—	postgres, redis, mysqld, nginx, blackbox, node, cAdvisor

Table 1: Services of the infrastructure.

Public entrypoints in production are exposed via `app.<domain>`, `lms.<domain>`, and `grafana.<domain>`; with the default `DOMAIN=localhost`, they resolve to `app.localhost`, `lms.localhost`, and `grafana.localhost` on port 443. Locally, the same names are served on port 8080 over HTTP (modern browsers resolve `*.localhost` to 127.0.0.1 automatically).

3 Images and design decisions

Frontend and caching. We use `nginx:alpine` as the single public entrypoint for all services. Nginx terminates TLS, routes subdomains to their internal services, and caches dynamic pages for 10 seconds and static files for one hour. When a user submits a review, a temporary cookie bypasses the cache so their new comment appears immediately. The page footer displays which frontend and application nodes handled the request, demonstrating load distribution across replicas.

TLS and certificates. Production exposes HTTPS on port 443 using a self-signed certificate generated at container startup, allowing Nginx to boot without external dependencies. The `letsencrypt` service is fully defined in the stack with automated renewal logic, but remains idle when running with local domains (`localhost`) where public certificate authorities cannot validate ownership.

Core application. The reviews application is built with Python 3.12 and runs as a non-root user with two replicas. To maintain high performance, page data is cached in Redis with version-based invalidation upon new submissions. The application degrades gracefully: if Redis is unreachable, requests fall back directly to the database; if the database is temporarily recovering, the service returns a clean HTTP 503 instead of crashing. Database operations are decoupled via HAProxy: writes target the leader (port 5432) while reads are distributed across replicas (port 5433).

PostgreSQL cluster (Patroni). Database high availability is managed by Patroni across three PostgreSQL 16 nodes (`pg-1`, `pg-2`, `pg-3`) distributed over `app1`, `app2`, and `stats`. A three-node etcd cluster provides distributed consensus for leader election and health tracking. If the active leader fails, Patroni automatically promotes a replica within seconds. HAProxy (`db`) sits in front of the cluster, continuously querying Patroni's health API to expose consistent endpoints (`db:5432` for writes, `db:5433` for reads) so application services require no configuration changes during a failover.

Redis cache and Sentinel. Redis caching uses a primary-replica architecture paired with three Redis Sentinels across separate nodes (quorum of 2). The application dynamically discovers the active master through Sentinel, enabling automatic failover if the primary node goes offline.

Moodle LMS. Moodle 4.5 LTS is deployed using PHP 8.3 Apache with two replicas. On initial launch, an automated script runs the command-line installer, utilizing a filesystem lock in the shared volume to prevent race conditions between containers. User sessions are persisted in MariaDB, allowing any replica to serve incoming requests interchangeably. A dedicated container (`moodle-cron`) runs scheduled tasks in the background.

Shared storage (NFS). Because Docker Swarm restricts privileged containers on worker nodes, we deploy **NFS-Ganesha** in user space on the `atenea` node. It exports shared volumes for Moodle data and certificates using only standard Linux capabilities, enabling seamless multi-node file sharing without requiring kernel-level NFS modules on the host.

Backups and observability. See Sections 7 and 8.

Scope of production. In the context of this project, "production" represents the complete multi-node Swarm architecture running on local Docker-in-Docker nodes. Because it is evaluated locally rather than on a public cloud provider, services use `localhost` subdomains and self-signed certificates.

4 Scalability and high availability

4.1 Replicas

The project guidelines require all services to be scalable, excluding periodic jobs and non-distributed DBMS. Stateless components (`frontend`, `core`, `static`, `db`, `smtp`, `grafana`, `moodle`, and `prometheus`) are deployed with two replicas and can be dynamically scaled using `docker service scale`. Prometheus automatically discovers active replicas via Docker Swarm DNS (`tasks.<service>`).

Services intentionally kept at a single replica include periodic background jobs (`letsencrypt`, `backup`, `moodle-cron`, and `auto`, where duplicate runs would be redundant), single-host storage (`nfs`), developmental tooling (`mailpit`), and `moodle-db`. The primary stateful layers (PostgreSQL and Redis) are not scaled naively, but are deployed as high-availability clusters with automated leader election and replication.

4.2 Failures

Table 2 summarizes the system resilience matrix, categorizing the direct impact, mitigation mechanism, and recovery process across service-level and node-level outages.

Failure Target	Direct Impact	Mitigation Mechanism	Recovery
Service / Container Failures			
Frontend replica	None	Swarm routing mesh directs traffic to survivor	Automatic
Core / static replica	None (transparent)	Nginx serves stale cache; Swarm reschedules	Automatic
Redis master	Brief cache miss spike	Sentinels promote replica; app degrades gracefully	Automatic (≈7–12 s)
Postgres leader	Writes pause until the election	Patroni elects replica; HAProxy reroutes traffic	Automatic (≈30 s)
Node Outages			
`front1` or `front2`	None	Routing mesh routes via surviving front node	Automatic
`app1` or `app2`	Reduced capacity	Other app node serves; Patroni/Redis failover; etcd 2/3 quorum	Automatic
`stats`	Metrics interrupted	Core app unaffected; etcd/Patroni maintain 2/3 quorum	On node restart
`atenea`	Moodle & NFS down	Core app unaffected; TLS certs cached in Nginx	On node restart
`backup`	Backup jobs pause	User-facing services completely unaffected	On node restart

Table 2: System resilience and failure recovery matrix.

4.3 Updates

Updates are applied one container at a time (`parallelism: 1`, `order: stop-first`) with automated rollback upon error (`failure_action: rollback`). Each custom image includes a defined `HEALTHCHECK`, guaranteeing that an updated replica is verified healthy before the deployment progresses to the next node. Furthermore, stateless services enforce `max_replicas_per_node: 1`, ensuring replicas reside on distinct physical hosts so that a single host failure never compromises service availability.

5 Deployment

We designed two deployment environments: a local environment for development and testing, and a production environment based on Docker Swarm. Both use the same images published in `omarcornejo/lab1grouph` on Docker Hub, so we can deploy the application without rebuilding the images. The main differences are how we organize the services, manage storage and expose the applications to users.

5.1 Local environment (test)

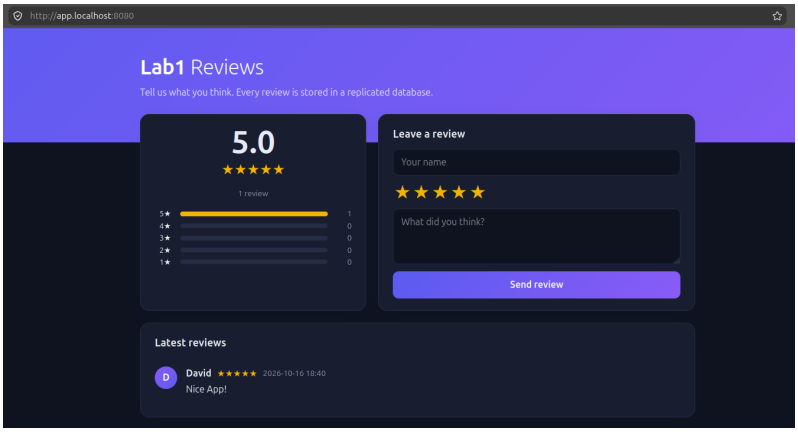
For the local environment, we used Docker Compose to run the entire infrastructure on a single machine. We chose this approach because it makes it easier to develop and test the different components without having to manage multiple nodes. Even though everything runs on one host, we kept the same main services as in production, including the PostgreSQL and Redis clusters, Moodle and the monitoring platform.

We defined the infrastructure in `compose.yml`, where we specified the services, networks and volumes. We used named volumes to keep persistent data even when containers are stopped or recreated. For simplicity, we configured the frontend to use HTTP on port 8080, since we do not need HTTPS certificates for local testing.

Figure 2 shows the local deployment. All 29 containers are running, and those with configured healthchecks report a healthy status. We also confirmed that the reviews application is accessible through `http://app.localhost:8080`.

```
SERVICE          STATE    HEALTH
auto             running  healthy
backup           running  healthy
blackbox         running  healthy
cadvisor         running  healthy
core             running  healthy
core             running  healthy
db               running  healthy
etcd-1           running  healthy
frontend         running  healthy
grafana          running  healthy
mailpit          running  healthy
moodle           running  healthy
moodle-cron      running  healthy
moodle-db        running  healthy
mysqld-exporter  running  healthy
node-exporter    running
pg-1             running  healthy
pg-2             running  healthy
pg-3             running  healthy
pg-exporter-1    running
pg-exporter-2    running
pg-exporter-3    running
prometheus       running  healthy
redis-1          running  healthy
redis-2          running  healthy
redis-exporter   running
sentinel         running  healthy
smtp             running  healthy
static           running  healthy
```

(a) Container status.



(b) Reviews application.

Figure 2: Local deployment: Docker Compose container status and application accessible at `http://app.localhost:8080`.

5.2 Production environment (simulated Swarm)

For production, we decided to use Docker Swarm to distribute the infrastructure across seven nodes. Three of them act as managers, while the other four are workers. Since we did not have seven

physical machines available, we used Docker-in-Docker to simulate the nodes on a single computer. This allowed us to work with a distributed architecture without needing additional hardware.

We defined the production configuration in `production.yml` and deployed it as a Swarm stack. Unlike the local environment, we used node labels and placement constraints to control where each service runs. This way, we could separate the frontend, application, monitoring, storage and backup services across different nodes.

We also configured multiple replicas for the services that need high availability, placing them on different nodes whenever possible. For the databases and cache, we kept the clustering mechanisms described in the previous sections.

To allow communication between services running on different nodes, we used Docker overlay networks. We configured two Nginx frontend replicas to handle incoming requests and forward them to the corresponding services. We also enabled HTTPS on port 443. Since we used `localhost` instead of a public domain, we used a self-signed certificate.

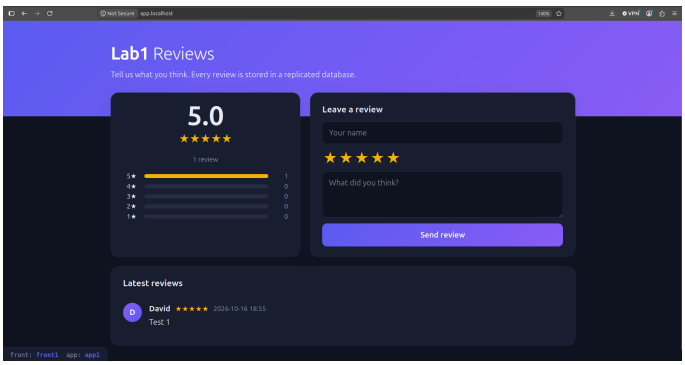
Figure 3 shows the production deployment. All seven nodes are in `Ready` and `Active` states, and the services have reached their expected replica counts. We also accessed the reviews application through `https://app.localhost`, where the footer shows which frontend and application nodes handled the request.

```
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
v0ggfbqmvmebbepkzj9xmsn5k     app1       Ready     Active         Reachable        27.5.1
w3ux8x1ra6pheqdf3tq0vtvkt     app2       Ready     Active                          27.5.1
jb28zhr00c2of59gxp711ly24     atenea     Ready     Active                          27.5.1
wkyazmqceq9mo521dfmt0f7y5     backup     Ready     Active                          27.5.1
cilezym9ut71gyv098r3429nl *   front1     Ready     Active         Leader           27.5.1
z49s1e3gsn4rhc4guhyje7tb1     front2     Ready     Active                          27.5.1
ov5ppq432a6t16y171wqmdg7g     stats      Ready     Active         Reachable        27.5.1
```

(a) Swarm node availability.

```
NAME                  MODE         REPLICAS
lab1_auto             replicated   1/1
lab1_backup           replicated   1/1
lab1_blackbox         replicated   1/1
lab1_cadvisor         global       7/7
lab1_core             replicated   2/2 (max 1 per node)
lab1_db               replicated   2/2 (max 1 per node)
lab1_etcd-1           replicated   1/1
lab1_etcd-2           replicated   1/1
lab1_etcd-3           replicated   1/1
lab1_frontend         replicated   2/2 (max 1 per node)
lab1_grafana          replicated   2/2
lab1_letsencrypt      replicated   1/1
lab1_mailpit          replicated   1/1
lab1_moodle           replicated   2/2
lab1_moodle-cron      replicated   1/1
lab1_moodle-db        replicated   1/1
lab1_mysqld-exporter  replicated   1/1
lab1_nfs              replicated   1/1
lab1_node-exporter    global       7/7
lab1_pg-1             replicated   1/1
lab1_pg-2             replicated   1/1
lab1_pg-3             replicated   1/1
lab1_pg-exporter-1    replicated   1/1
lab1_pg-exporter-2    replicated   1/1
lab1_pg-exporter-3    replicated   1/1
lab1_prometheus       replicated   2/2 (max 1 per node)
lab1_redis-1          replicated   1/1
lab1_redis-2          replicated   1/1
lab1_redis-exporter   replicated   1/1
lab1_sentinel-1       replicated   1/1
lab1_sentinel-2       replicated   1/1
lab1_sentinel-3       replicated   1/1
lab1_smtp             replicated   2/2 (max 1 per node)
lab1_static           replicated   2/2 (max 1 per node)
```

(b) Service replica status.



(c) Production application.

Figure 3: Simulated production deployment: Swarm nodes, service replicas and application accessible at `https://app.localhost`.

In this section, we focused on showing how we deployed both environments and how we organized the infrastructure. We cover failure recovery, backups and scalability in Section 10.

6 Access, Users and Passwords

For security and centralized configuration, credentials are injected with Docker secrets mounted at `/run/secrets`. For testing purposes within this laboratory setup, default passwords are standardized to `groupH`, the only exception is the Moodle administrator, because Moodle refuses passwords that do not follow its policy (upper and lower case, digit, and symbol).

6.1 Web Addresses

Service	Local (`compose.yml`)	Production (`production.yml`)	Auth
Reviews application	`http://app.localhost:8080`	`https://app.localhost`	Public
Moodle (Atenea)	`http://lms.localhost:8080`	`https://lms.localhost`	Required
Grafana	`http://grafana.localhost:8080`	`https://grafana.localhost`	Required
Mailpit	`http://mail.localhost:8080`	`http://mail.localhost:8080`	Public

Table 3: Deployed web endpoints and authentication requirements.

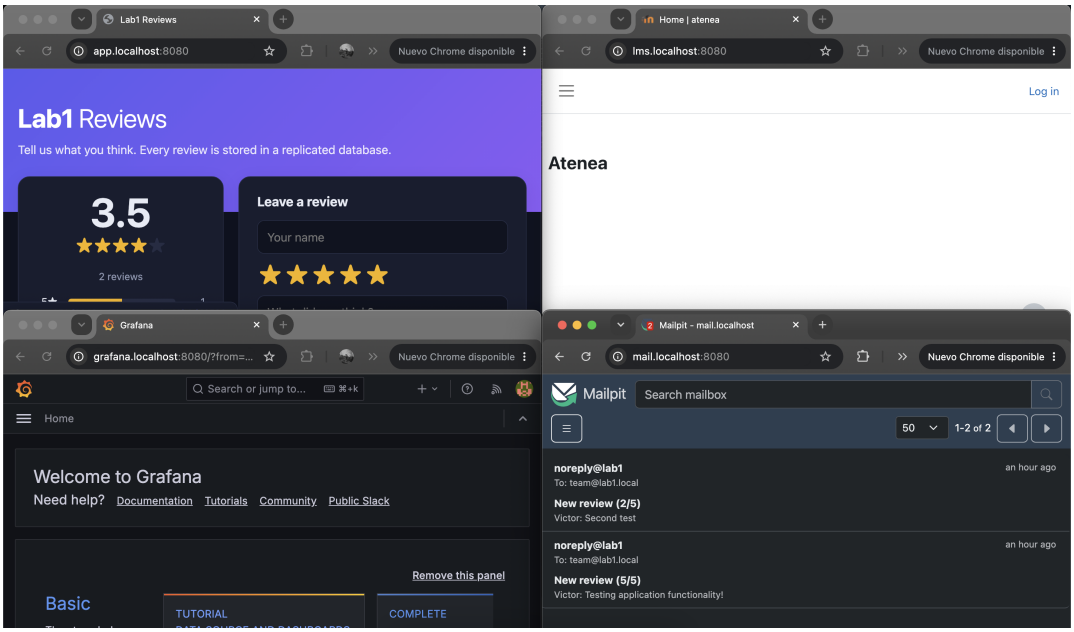


Figure 4: Running deployment of the web addresses.

6.2 Users and Passwords

Table 4 shows the service credentials, secret files, and the configured database access scopes.

Service	User	Password	Secret File
Grafana	admin	groupH	grafana_admin_password
Moodle	admin	groupH-2026	moodle_admin_password
PostgreSQL (Patroni)	postgres	groupH	pg_admin_password (Internal HA management)
PostgreSQL (App)	app	groupH	app_db_password (Owner of app DB)
PostgreSQL (Grafana)	grafana	groupH	grafana_db_password (Owner of grafana DB)
PostgreSQL (Replication)	replicator	groupH	replicator_password
PostgreSQL (Backup)	backup	groupH	backup_db_password (pg_read_all_data)
PostgreSQL (Metrics)	exporter	groupH	exporter_db_password (pg_monitor)
MariaDB (Moodle)	moodle	groupH	moodle_db_password
MariaDB (Auxiliary)	backup, exporter	groupH	Shared secrets (Read-only access)
Backup Encryption	—	groupH	backup_passphrase (AES-256-CBC)
Self-signed TLS key	—	groupH	CERT_PASSWORD
Feeder API	—	groupH	feed_token (Bearer Token)

Table 4: System account roles and Docker secret mappings.

7 Data Management and Backups

To avoid database corruption caused by network disruptions or file-locking conflicts, transactional DBMS files are stored directly on local node drives rather than over NFS.

Data Target	Volume Name	Physical Node Location
PostgreSQL Cluster & etcd	pg-1..3, etcd-1..3	Local disk (app1, app2, stats)
MariaDB & NFS Root	moodle-db, nfs-data	Local disk (atenea)
Moodle Data & Certificates	moodledata, certs, acme	NFS Storage (atenea)
Encrypted Backups	backups	Dedicated disk (backup)
Prometheus Replicas	Anonymous volumes	Local disk (stats, backup)
Redis Cache	In-Memory	Replicated RAM

Table 5: Physical storage distribution across Swarm nodes.

The backup service operates on a dedicated host isolated from core application nodes. Scheduled daily at 03:30, the pipeline performs:

1. **PostgreSQL Dump**: Uses `pg_dump -Fc` routed to read-only replicas (`db:5433`) under the `backup` role to prevent load on the HA leader.
2. **MariaDB Dump**: Executes `mariadb-dump -single-transaction` for transactional consistency.
3. **Moodle Data Storage**: Archives `moodledata` while stripping volatile session and cache directories.
4. **Encryption & Integrity**: Encrypts files using OpenSSL AES-256-CBC (PBKDF2) with the secret passphrase and generates matching `SHA256SUMS`.

Backups are initially written with a `.partial` suffix and atomic-renamed upon completion, and retention is configured for 14 days.

An automated integrity check runs every Sunday (and on demand with `docker exec <backup> /restore-test.sh`). The routine verifies checksums, decrypts archives, restores dumps into temporary containerized DBMS instances, and validates recovered row and table counts. Test outcomes are exported to Prometheus (`backup_restore_test_ok`).

To recover data, `scripts/restore.sh <folder> app|grafana|moodle|moodledata` is run on the backup node: it decrypts the chosen file of a backup folder and loads it into the live service (for PostgreSQL, through `db:5432`, that is, into the current leader).

8 Observability

Prometheus uses DNS service discovery (`tasks.<service>`) so scaled replicas are scraped without manual changes. Dashboards are provisioned declaratively via Git, where Grafana visualizes node metrics alongside custom ratings using the Infinity plugin. Additionally, Prometheus alerting rules (`rules.yml`) flag offline endpoints, replication delays, missing cluster leaders, and backup failures. Table 6 summarizes the metric sources and monitored targets.

Metric Source	Monitored Target	Key Operational Indicators
`core`	Python Microservice	Requests/sec, posted reviews, Redis cache hit/miss ratio
Nginx Exporter	Frontend Proxy	Active connections, throughput per replica
Postgres Exporter / Patroni	PostgreSQL Nodes	Connection pool, transactions, replication lag, leader status
Redis Exporter	Cache Instances	Master/Replica role, memory consumption, hit ratio
MySQLd Exporter	MariaDB Instance	Active threads, queries per second (QPS)
Blackbox Exporter	Web Endpoints	Availability and HTTP response latency for Moodle/Grafana/App
cAdvisor & Node Exporter	Host Nodes	Node-level CPU, RAM usage, and container resource limits
Backup Exporter (9199)	Backup Service	Last execution age, duration, archive size, restore status

Table 6: Observability exporters and monitored metrics.

Figure 5 presents the main Grafana overview dashboard after simulating application traffic generated by user interactions with the reviews app.

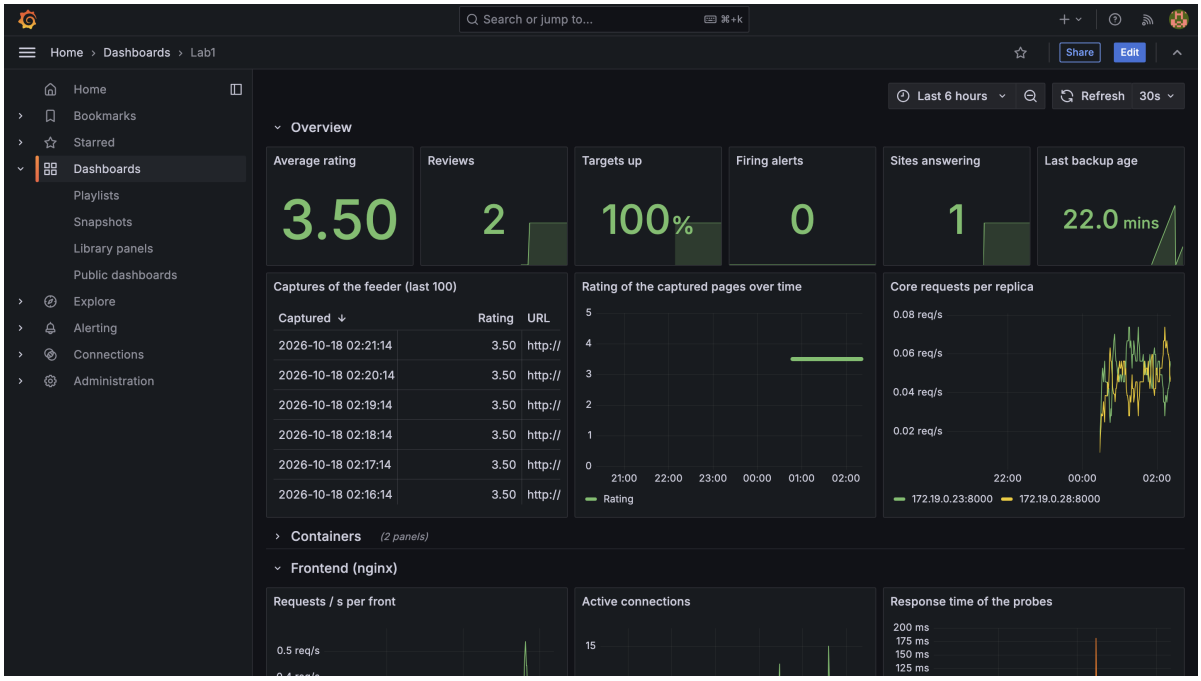


Figure 5: Grafana monitoring dashboard.

9 Security

- **Traffic Control**: Only Nginx exposes external ports (80/443). All traffic is forced through TLS 1.2/1.3 with HSTS enabled. Internal microservices remain isolated.

- **Least-Privilege Database Access**: Database owners (`app`, `grafana`) are restricted to their respective databases. The `replicator` user is confined to replication streams, and global `CONNECT` rights are revoked from `PUBLIC`.
- **Encrypted Inter-Node Communication**: Secrets are mounted in-memory at `/run/secrets`. Overlay networks utilize IPsec encryption, segregating individual application stacks.
- **Metrics Isolation**: Metric endpoints (`/metrics`) are blocked at the proxy layer, preventing external exposure.

10 Test cases

The basic functionality (reviews, mails, Moodle and monitoring) is already shown in the previous sections, so here we focus on what happens when something fails, on backups and on scaling. Every test was run on a clean deployment, first locally and then in production, and in every case we checked that the reviews application kept working. Table 7 summarises them; the commands and the important lines of their output follow.

#	Test	What we expect	Result
L1	Database leader crashes	another node becomes leader, no review is lost	passed (30 s)
L2	Cache master stops	the replica becomes master, the page keeps working	passed (12 s)
L3	Backup and restore test	the backup can really be restored	passed
L4	Three app replicas	traffic is shared among the three	passed
P1	A whole node goes down	the site stays up, the database elects a leader	passed (31 s)
P2	Cache master is killed	the three Sentinels promote the replica	passed (7 s)
P3	Restore the database	data goes back to the moment of the backup	passed

Table 7: Test cases and results.

10.1 Local environment

L1: The database leader crashes

We looked at which PostgreSQL node was the leader (`pg-3`), published two reviews and killed it as if it had crashed.

```
$ docker compose kill pg-3
$ docker compose logs pg-1 pg-2 | grep promoted
pg-2 | 14:09:18 INFO: promoted self to leader by acquiring session lock
$ docker compose exec pg-1 patronictl -c /etc/patroni.yml list
| pg-1 | Replica | streaming | TL 2 |
| pg-2 | Leader | running | TL 2 |
| pg-3 | Replica | streaming | TL 2 | # after starting it again
```

After about 30 s (the time the leader has to renew its lock) `pg-2` became the new leader. The application did not notice it, because it always connects through HAProxy, which follows the leader. Two more reviews were published after the failure and all four were in the table. When `pg-3` was started again it caught up with the new leader and came back as a replica by itself.

L2: The cache master stops

`redis-1` was the Redis master and `redis-2` its replica. We stopped the master.

```
$ docker compose stop redis-1
$ docker compose logs sentinel | grep switch-master
14:22:23 +switch-master mymaster redis-1 6379 redis-2 6379
$ docker compose start redis-1
$ docker compose exec redis-1 redis-cli info replication
```

```
role:slave
master_host:redis-2
```

Sentinel noticed the failure after 5 s and made `redis-2` the new master about 7 s later. Meanwhile the page kept loading and accepted a review, since the application asks Sentinel where the master is and can work without cache. When `redis-1` came back, Sentinel turned it into a replica of the new master.

L3: Backup and restore test

We counted the reviews (6), took a backup by hand and ran the restore test on it.

```
$ docker compose exec backup /backup.sh
backup ok: /backups/20261018-1426 (474910 bytes, source: db:5433)
$ docker compose exec backup /restore-test.sh
app.dump.enc: OK grafana.dump.enc: OK
moodle.sql.gz.enc: OK moodledata.tar.gz.enc: OK
restore test ok: 6 comments, 494 moodle tables
```

The backup was read from a replica (`db:5433`), so it did not load the leader, and it contains the encrypted copies of the two PostgreSQL databases, the Moodle database and moodledata. The restore test checked the files, decrypted them and loaded them into temporary databases: it got back the same 6 reviews and the 494 Moodle tables, so the backup is not only present but usable.

L4: Three replicas of the application

We scaled the application to three replicas and sent 12 requests (skipping the nginx cache, so that each one really reaches the application).

```
$ docker compose up -d --scale core=3
$ for i in $(seq 12); do curl ... | grep x-app-node; done | sort | uniq -c
      6 X-App-Node: 3296c6cf48d3
      2 X-App-Node: 7ad5cb8834d9 # the new replica
      4 X-App-Node: f8a2281d16e7
```

The new replica started receiving requests a few seconds after it was created, without restarting nginx, because nginx looks up the name `core` again every 10 s. This is possible because the application keeps no data of its own: reviews are in PostgreSQL and the cache in Redis. Going back to two replicas did not affect the page.

10.2 Production environment

In production the commands run inside the simulated nodes (`docker exec <node> ...`); `pctl`, `sent` and `pcount` are short helpers that run `patronictl`, the Sentinel client and a count of reviews in a node that is not affected by the test.

P1: A whole node goes down

The node `app2` runs one replica of the application and one member of each cluster (PostgreSQL, Redis, etcd and Sentinel). To also test the election, we first moved the database leadership to `pg-2`, on that node, and then switched the node off.

```
$ pctl switchover --leader pg-3 --candidate pg-2 --force
$ docker stop app2 # 14:45:58
$ docker exec front1 docker node ls
app2 Down Active
$ pctl list
| pg-1 | Leader | running | TL 3 | # promoted at 14:46:29
| pg-3 | Replica | running | TL 2 |
```

```
$ curl -sk -o /dev/null -w '%{http_code}\n' https://app.localhost/
200
```

Swarm saw the node as *Down* within seconds and 31 s after the stop `pg-1` became the leader; the two surviving etcd members are still a majority, which is what allows the election. The page answered all the time through the application replica on `app1`, and a review published after the election was saved. When `app2` was switched on again, swarm restarted its containers and `pg-2` rejoined as a replica.

P2: The cache master is killed

The Redis master was `redis-1` on `app1`, with three Sentinels on `app1`, `app2` and `stats`. We killed the container.

```
$ docker exec app1 sh -c 'docker kill $(docker ps -qf name=lab1_redis-1)'
                                    # 14:52:13
$ docker exec front1 docker service logs --raw lab1_sentinel-1 | grep -E ...
14:52:18 +odown master mymaster redis-1 6379 #quorum 2/2
14:52:20 +switch-master mymaster redis-1 6379 redis-2 6379
$ sent sentinel get-master-addr-by-name mymaster
redis-2
```

The three Sentinels agreed that the master was down 5 s after the kill and `redis-2` was promoted 1 s later, about 7 s in total. The page kept answering and accepted a review. Swarm restarted `redis-1`, and Sentinel turned it into a replica of `redis-2`.

P3: Restore the database from a backup

This simulates recovering from a mistake: we took a backup with 3 reviews, published a fourth one and restored the backup on the backup node.

```
$ pcount # 3
$ bk /backup.sh
backup ok: /backups/20261018-1456 (444014 bytes, source: db:5433)
$ pcount # 4, review published at 14:57:16
$ docker exec backup sh /restore.sh 20261018-1456 app
$ pcount # 3
$ ... psql -c "select id, author, created from comments" app
  1 | test 1 | 14:45:51
 34 | test 2 | 14:46:53
 35 | test 3 | 14:53:03
$ docker exec front1 docker service logs --raw lab1_core | grep 'POST /comment'
... 14:53:03 "POST /comment HTTP/1.1" 302
... 14:57:16 "POST /comment HTTP/1.1" 302 # the review that disappears
```

The logs and the notification mail in Mailpit show that the fourth review existed (14:57:16), and after the restore only the three reviews written before the backup are left, so the database went back exactly to the moment of the backup. The tables keep their owner, so the application kept working on the restored data and, once its caches expired (about 40 s), the page showed only those three reviews. The jump from id 1 to 34 is not a lost review: after the failover of P1 PostgreSQL skips the ids that the old leader had reserved.

11 Results

11.1 Local environment

The stack is started with `docker compose pull` and `docker compose up -d` and its 29 containers become healthy in a few minutes (Moodle is the slowest, because it installs itself on the first run). In

the tests, the loss of the PostgreSQL leader was solved by Patroni in about 30 s and the loss of the Redis master by Sentinel in about 12 s; in both cases the page kept working and no review was lost. The backup is encrypted and the restore test recovers the reviews and the 494 Moodle tables. The application scales to three replicas and the traffic is shared among them without touching nginx.

11.2 Production environment (`production.yml`)

Production runs the same stack on seven nodes, simulated on a single computer with Docker-in-Docker containers (`scripts/sim-swarm.sh`), three of them acting as swarm managers. After the deployment the 7 nodes were *Ready* and the 34 services had all their replicas. The tests gave the following results:

- When the node `app2` is switched off, the site stays up through the other application node, a new database leader is elected in about 30 s and new reviews are saved again. When the node comes back, its services return by themselves.
- When the Redis master is killed, the three Sentinels promote the replica in about 7 s and the old master returns as a replica.
- A backup taken on the backup node, from a database replica, brings the application database back to the moment it was taken, and the application keeps working on it.

12 Limitations

- Production is simulated on one machine, so real network latency between nodes was not measured. It is not exposed to the Internet and has no public domain, so the certificate is self-signed (the Let's Encrypt configuration is included but unused).
- PostgreSQL and etcd need two of their three nodes. If two fail together, writes stop until one recovers. The third PostgreSQL node also hosts the monitoring.
- MariaDB, the NFS server and the mail catcher run as single instances, so losing the atenea node stops Moodle until it returns.
- Backups are stored on a single disk with no off-site copy.
- For simplicity, all secrets are `groupH` and are committed to the repository, NFS uses `no_root_squash` (restricted to private networks, its port should be firewalled) and the mail catcher has no authentication.

13 Conclusions

We turned a company infrastructure into a set of containerised services that can be deployed with a couple of commands, either on one machine for development or on seven nodes for production. The system includes the reviews application, Moodle, monitoring, mail and encrypted backups, all published as images.

The hardest part was not getting the services to start but making them tolerate failures: service discovery, leader election and automatic recovery. Many issues only showed up when we deliberately stopped nodes, so testing failures early was essential. In the final system, losing a web server, an application server, the cache or the database leader is transparent to the user, and backups are verified by automatic restores.

The deployment still has the simplifications listed above, but its architecture would be the same on real hardware. Future work would be deploying it on real servers with a public domain, keeping off-site backups and proper secret management.